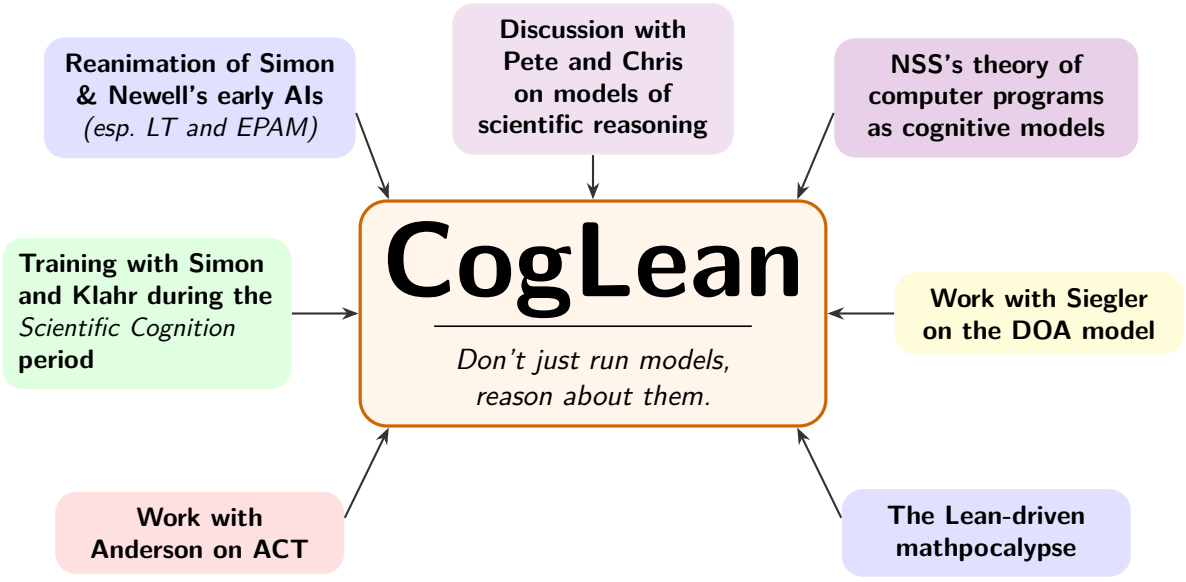
Epoofany


Reanimation of Simon & Newell's early AIs (esp. LT and EPAM)
Discussion with Pete and Chris on models of scientific reasoning
NSS's theory of computer programs as cognitive models
Training with Simon and Klahr during the Scientific Cognition period
CogLean
Don't just run models, reason about them.
Work with Siegler on the DOA model
Work with Anderson on ACT
The Lean-driven mathpocalypse

Composing Cognitive Theories in CogLean

Multi-Digit Addition

Jeff Shrager

November 2026

Draft for Comment

CogLean, in one slide

- A library of cognitive theory in Lean, organized like Mathlib: **mechanisms** defined once; **phenomena** stated as predicates with no mechanism in them (targets); **theorems** relating them, under explicit **conditions**; **laws** for whole classes of mechanisms.
- Theorems are **explanations**: deductions of a phenomenon from mechanisms and assumptions.
- They are proved about **the same program that runs**: change the model, and every explanation it breaks fails to check.
- A value at one setting is a **witness**, not an explanation.

Can independently developed theories, in a common formal library, be composed to derive meaningful consequences we'd otherwise need a new model to find?

Chris's question

Chris, a hypothetical modeler who studies children's arithmetic, works with **Robin**, a chatbot colleague: good at finding library results and suggesting proofs, *not* automatically right.

- Children sometimes confidently answer $150 + 100$ with 200. Does that follow from existing mechanisms, and under what conditions?
- Answering it means **composing** two independently formalized theories with a problem solver, and **reusing their theorems**.

Then: what's under the hood of the composition, and what it does and doesn't show.

Part 1

From an observation to a theorem

Does confident error follow from mechanisms we already have?

Chris's observation

- Children sometimes **confidently** give a wrong answer, apparently retrieving it rather than calculating.
- A child who knows $100 + 100 = 200$ takes $150 + 100$ for the familiar problem, and says **200**, without hesitating.
- Chris's question: does this follow from recognizable properties of *existing* mechanisms?

Robin: CogLib has a general mechanism for exactly this shape,

recognize a stimulus $\rightarrow$ retrieve an associated response $\rightarrow$ back up if retrieval fails,

and two theories already formalized that fill it in.

Two theories, formalized on their own

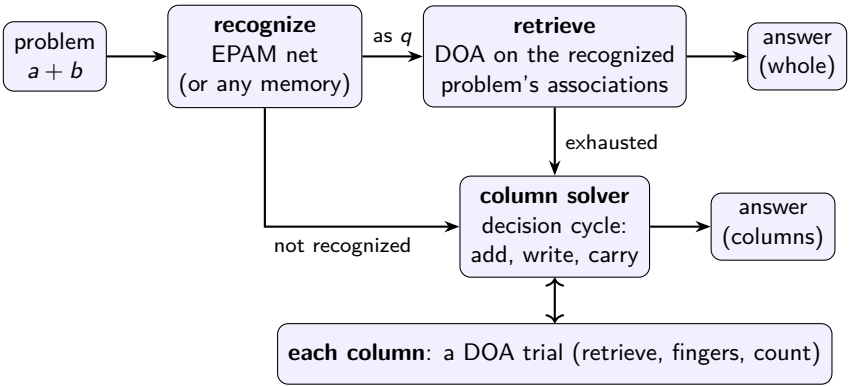
EPAM (Feigenbaum & Simon)

- A discrimination net sorts a code to a stored image.
- Learning grows a test only where codes are confused.
- Proved for every net: recognition depends only on tested features; learning never disturbs other items.

DOA (Siegler & Shrager 1984)

- Associations from problems to answers; sample, state if above a criterion; else fingers, then counting.
- Proved for every setting: retrieval used with probability $1 - (1 - R)^n$; counting errors independent of the associations.

The application: three-digit addition



A demonstration, not a theory of arithmetic. In Lean, the composition is three definitions.

The phenomenon, stated without a mechanism, and when it follows

If q is recognized as itself, x is confused with q, q's associations are **sharp**, and their sums differ:

(i) whole-problem retrieval never errs on q;

(ii) every whole-problem answer to x is wrong;

(iii) x reaches the solver *exactly as often* as q.

- **Neither theory alone**: in DOA, errors and backup use come from one distribution; recognition gives x another problem's distribution.
- **Reused**: a one-line proof, from CogLib's theorem for *any* task with problems, a familiar set and a backup.
- **Conditions**: confusion (derived for learned nets); sharpness (assumed, not yet explained); positive strengths.

In words: the theorem on the next slide

CogLib's `confident_errors`, for *any* task, memory and backup:

- **Parameters**: how problems are coded, what memory calls up (`retrieve`), the associations (`str`), the criterion and search length, and the backup.
- **Hypotheses**:
  - strengths have positive totals; the backup is a proper distribution;
  - q calls up itself, for certain (`hq`);
  - x calls up q, for certain (`hx`): the confusion;
  - q's associations are sharp (`hs`); the correct answers differ (`hne`).
- **Conclusion**: the *confusion-error pattern*, a phenomenon defined once in CogLib, apart from any mechanism: three equations, (i)–(iii) of the last slide.
- **Proof**: three lines, citing three earlier lemmas.

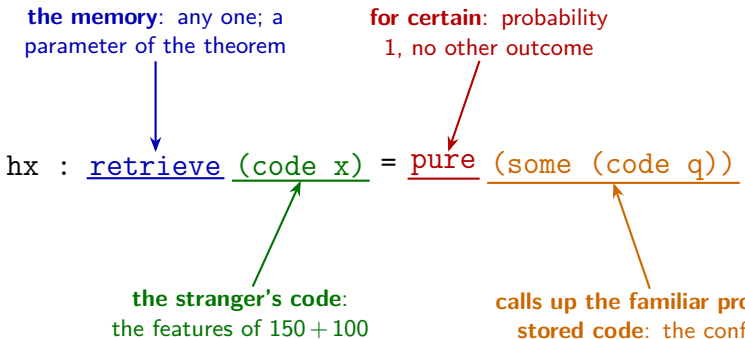
The theorem, in Lean

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (recognizeRetrieve code retrieve str as cc n backup)
      (wholeWrong correct) isWhole isBackup x q := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup
      correct q q (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup
      correct x _ _ (hT _) hx hs hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]

def ConfusionErrorPattern (trial : P → Dist O) (wholeWrong : P → O → Bool)
    (isWhole isBackup : O → Bool) (x q : P) : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧
    prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧
    prob (trial x) isBackup = prob (trial q) isBackup
```

Verbatim from CogLib (`Process/RecognizeRetrieve`, `Phenomena/ConfusionErrors`). The multi-digit version is a one-line instance.

One clause, unpacked: the confusion



- **EPAM**: holds for the net learned from $100 + 100$ and $100 + 200$ (checked); every learned net recognizes its own images.
- **ACT-R**: *never* holds, because `pure` is too strong: retrieval is graded, never certain. That is why the theorem doesn't apply to it.

Equal is not low

- The theorem gives *equal* solver use for x and q, not *low* solver use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the confused problem reaches the solver with probability at most $(1 - r)^n$.
- With flat associations on q, the same theorem holds, and the errors are not confident.

Here $R_q = 45/46$, $n = 1$: the solver is reached $1/46$ of the time.

Does it need EPAM? Swapping the memory

	EPAM net	ACT-R memory
$150 + 100$ answered 200 by retrieval	$45/46$	$225/529 \approx .43$
$150 + 100$: solver invoked	$1/46$	$383/1058 \approx .36$
solver correct, *given* invoked	$\approx .656$	$\approx .656$
correct overall (invoked *and* correct)	$\approx .014$	$\approx .238$
practiced $100 + 100$: overall errors	$\approx .0073$	$\approx .334$
after also studying $100 + 300$	unchanged	$\approx .616$
memory laws (every setting)	determinacy, certainty, stability	none of them

- The confident-error theorem holds for *any* memory with the three laws.
- For ACT-R it doesn't apply. That doesn't mean ACT-R can't err similarly (it does, gradedly); the exact pattern fails, and at this setting that is proved (`setting_actr_not_pattern`).

ACT-R, primed: a second explanation

- A **graded** pattern needs no certainty: if x calls up q, and q itself, each with probability $\geq 1 - \delta$, the pattern holds within 2δ; graded laws give it for a class.
- ACT-R reaches it when q is **primed**; for every $\delta > 0$, enough priming suffices. Having *just seen* q is such priming, through ACT-R's base level.

	$150 + 100$ answered 200 by retrieval
ACT-R, unprimed	$225/529 \approx .43$
ACT-R, $100 + 100$ primed $\times 10$	$2250/2599 \approx .87$
ACT-R, primed $\times 100$	$22500/23299 \approx .97$
EPAM	$45/46 \approx .98$

Two explanations: discrimination, and primed partial matching. Priming also makes *other familiar* items taken for q; EPAM never does that.

What Chris has now

- Not a simulation of one error, but a **reusable result**: sufficient conditions for the phenomenon, for every setting that meets them.
- It applies to a new model **without re-running** the old one, and to *any* memory with three laws.
- The conditions are where the science is:
  - Lean hasn't shown that children have sharp associations, or that a given learning history produces the confusion: separate theoretical or empirical questions;
  - the pattern by itself doesn't make backup use rare; *confident* needs a further, quantitative condition.

Part 2

Under the hood

How the composition in part 1 runs, and why its theorems transfer

What actually happens when retrieval fails

1. **Recognize**: the EPAM net sorts the problem's code to a stored problem, possibly the wrong one.
2. **Retrieve**: DOA searches that problem's associations for an answer above the criterion.
3. **Fail**: if the search is exhausted, the backup is invoked.
4. **Solve**: the backup runs column addition on CogLib's general problem-solving machinery.
5. **Each column**: units, then tens, then hundreds; each digit sum is a separate DOA trial (retrieve, else fingers, else count).
6. **Combine**: each trial's *distribution* of answers feeds the rest of the calculation, with carrying and place value.
7. **Result**: a probability distribution over complete multi-digit answers, errors included.

`Dist.bind` composes these steps: it carries *every* possible outcome, with its probability, through the steps that follow. Nothing is sampled, and nothing looks up the right answer: correct knowledge enters only as single-digit associations and the carry rule.

The general machinery, and its arithmetic instance

CogLib (`Process/ProblemSolving`), task-general:

- problem spaces: states; operators with preconditions and (probabilistic) effects;
- a **decision cycle**: a procedure proposes; a violated precondition is an *impasse*, fixed by repairs and critics;
- separately, **GPS**: goals, subgoals, backtracking.

MultiDigit (`Solver.lean`), the instance:

- operators: add a column, write, write and carry, finish;
- `full`: the procedure that proposes them in turn;
- `lenient`: the repair policy (no critics).

```
def solverBackup (F : Facts) (p : Problem) : Dist Nat :=
  Solver.answers F.fact Solver.full Solver.lenient p
```

`F.fact`: the DOA single-digit trial; `Solver.answers`: runs CogLib's decision cycle. The integrated model uses the **decision cycle**; GPS runs on the same space in `SolverGPS.lean` (units first; exact with exact facts) but is not part of it.

The bridge: two executable descriptions, one distribution

- `columns`: a simple, direct implementation of probabilistic column arithmetic.
- `answers`: the same task on CogLib's state-and-operator decision cycle.

```
theorem bridge (fact : Nat → Nat → Dist Nat) (p : Problem) :
    answers fact full lenient p = columns fact p
```

- **Proved**, not tested: identical answer distributions for *every* problem and *every* single-digit process.
- So every theorem about `columns` (place value, a column error as a single-digit error in place) holds for the solver.
- It equates *outputs*, not internal processes: the two still differ as accounts of what a child does.

How theorems compose

- **The bridge** (previous slide): every column theorem transfers to the solver.
- **Associativity**: every arrow is a two-stage draw; draws regroup, so a component's theorem holds inside the whole.
- **Well-formedness, inherited**: the composed trial is a probability distribution, proved by chaining lemmas from three packages:
  - retrieval, a DOA trial, the column procedure, the bridge;
  - total mass 1, *and* nonnegative weights.

These are library facts, not psychology; they are what let results transfer.

Two more consequences of the composition

Whole-problem errors don't depend on the solver (every setting)

- Improving column skill cannot reduce errors from mistaken recognition.

Two sources of error on a confused problem (given the conditions)

- Whole route: always the neighbour's sum.
- Column route: exhaustion $\times$ the solver's own error rate, which comes from the single-digit DOA trials.
- If the other columns are exact, a column error is a single-digit error in place, off by $10^j\delta$. But columns can err together, errors can carry, and they can even cancel: units 10 and tens 4 still give 250.

One memory for both levels

Column facts from the *same* memory as whole problems: each digit pair $d + e$ is coded as a problem, recognized, and retrieved in the same strength table; fingers and counting if that fails.

- **The old model is a special case**: a pair recognized as itself gives exactly the DOA fact (`memFact_self`), so every earlier theorem carries over.
- **Confident errors one level down**: a pair taken for another states the other's sum (`column_confident_errors`), with no new proof.
- **Noninterference needs a condition**: EPAM's practice stays undisturbed *given* that the problem's digit pairs were learned (`shared_learning_preserves_trial`). The old theorem held the solver fixed; here learning could change it.

One memory: the numbers

Memory studies $0+0$, $5+0$, $1+1$, then $100+100$, $100+200$.

	EPAM	ACT-R, separate table	ACT-R, one memory
$0+0$ answered 5 (retrieves $5+0$)	0	0	120/2167
$150+100$: solver invoked	1/46	$\approx .407$	$\approx .407$
solver correct, *given* invoked	$\approx .656$	$\approx .656$	$\approx .385$
correct overall	$\approx .014$	$\approx .267$	$\approx .157$
errors on practiced $100+100$	$\approx .0073$	$\approx .334$	$\approx .386$

- The middle column is a control: same five items in memory, but the solver doesn't use it.
- Extra items alone: *more* correct ($.238 \rightarrow .267$). Sharing the memory: fewer (.157).
- ACT-R without retrieval as use; finger counting is an implausible column fallback.

Part 3

What this shows, and what it doesn't

Checks, kinds of results, limits

What each result rests on

Kind	Examples
Library facts	the bridge; the composed trial is a probability distribution
Compositional deductions (every setting)	whole-problem errors independent of the solver; solver use is $(1 - R_q)^n$
Conditional	confident errors; two sources of error; any memory with three laws
At chosen settings	$45/46$, $1/46$, $\approx .656$, $\approx .014$, the ACT-R column

None of these is yet an empirical finding about people. The numbers are illustrations, not fits.

Checking the formalization: a blind simulation

- A prose specification, no Lean; preregistered checks; a fresh agent wrote a simulator from the prose alone.
- **Agreement**: every check whose hypothesis arose passed; 243 frequencies matched Lean exactly.
- **Found what proof didn't**:
  - five theorems' hypotheses were *unrealized* in the tested learning histories (not shown unreachable);
  - a gap at zero total strength;
  - a predicted learning sequence depends on an unstated assumption: a new problem's initial associations.

Limitations, and next

Limitations

- Strengths chosen, not learned; sharpness assumed.
- Error-free carrying; independent columns; finger counting as the column fallback.
- ACT-R without retrieval as use.
- Nonnegativity proved for this model, not library-wide.
- Scalability: exact finite distributions.
- No contact with data yet.

Next

- Derive associations from learning histories, so sharpness is a theorem.
- Learning across whole trials: prove the learning sequence.
- Restate the unrealized predictions in terms of tested features.
- Retrieval that changes memory, carried through the solver.
- Architectural constraints; other domains.

Take-home

- Cognitive theories can be stated once, in a shared formal library, with their consequences proved.
- Independently formalized theories compose, and so do their **theorems**.
- Composition derives consequences none of the parts states; formalization exposes what the theories didn't say.
- Every result is labelled: library fact, compositional deduction, conditional, or illustrative.
- Theorems are **explanations**, proved about the programs that run, so they stay aligned with the models as both change.

Not formalizing proofs, but formalizing explanations, so that we can reason about them. The goal: a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and traditions.

`https://github.com/jeffshrager/CogLean`